

系统开发工具基础

第 3 周课上实验报告

课程名称： 系统开发工具基础

授课主题： Packaging and Shipping Code; 智能体编程;
不止于代码; Python 与 PyTorch

授课日期： 2026 年 9 月 7 日（周一，第 5—8 节）

姓 名： 张鑫硕

学 号： 25020007170

GitHub: <https://github.com/ADayxs/SystemDevelopmentTools.git>

September 7, 2026

Contents

1	实验概览	2
2	Q09：从源码构建并在干净环境安装 Wheel	2
2.1	题目要求	2
2.2	实现步骤	2
2.2.1	1. 创建项目结构	2
2.2.2	2. 构建 Wheel	3
2.2.3	3. 创建干净虚拟环境并安装	3
2.2.4	4. 验证输出	3
2.3	截图记录	3
2.4	GitHub Commit 记录	3
3	Q10：让编程智能体进入可验证的修复循环	3
3.1	题目要求	4
3.2	实现步骤	4
3.2.1	1. 复制项目并添加失败测试	4
3.2.2	2. 运行测试确认失败	4
3.2.3	3. 向编程智能体描述目标	4
3.2.4	4. 智能体修改后的核心 Diff	4
3.2.5	5. 再次运行测试	5
3.3	截图记录	5
3.4	ai_log.md 记录	5
3.5	GitHub Commit 记录	5
4	Q11：把”无法处理”的协作材料改成可执行信息	5
4.1	题目要求	5
4.2	改写结果	5
4.2.1	1. Issue（重写）	5
4.2.2	2. 提交信息（重写）	6
4.2.3	3. 评审意见（重写）	6
4.3	字数统计	6
4.4	GitHub Commit 记录	6
4.5	截图记录	6
5	Q12：修复一个可复现的线性回归训练循环	6
5.1	题目要求	7
5.2	完整代码实现	7
5.3	核心补全说明	7
5.4	运行结果	7
5.5	截图记录	8
5.6	GitHub Commit 记录	8
6	实验总结	8

1 实验概览

本次实验共 4 道题目，涵盖以下主题：

题号	主题	内容概要	用时
Q09	Packaging and Shipping Code	从源码构建 Wheel 并在干净环境安装	12–15 min
Q10	智能体编程	测试驱动修复：空白姓名处理	12–15 min
Q11	不止于代码	改写协作材料为可执行信息	12–15 min
Q12	Python 与 PyTorch	修复线性回归训练循环	12–15 min

2 Q09：从源码构建并在干净环境安装 Wheel

2.1 题目要求

在 q09 中按给定内容创建一个最小 Python 命令行包 greetlab，并验证 wheel 安装。

2.2 实现步骤

2.2.1 1. 创建项目结构

按题目要求创建如下文件：

Listing 1: 项目目录结构

```
q09/
├── pyproject.toml
├── src/
│   └── greetlab/
│       ├── __init__.py
│       └── cli.py
```

src/greetlab/__init__.py 为空文件。src/greetlab/cli.py 内容如下：

Listing 2: cli.py

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

pyproject.toml 中将 greetlab-学号替换为实际学号：

Listing 3: pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007170"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

2.2.2 2. 构建 Wheel

在 q09 目录下运行：

```
python -m build
```

构建成功后在 dist/ 目录下生成 .whl 文件。

2.2.3 3. 创建干净虚拟环境并安装

Listing 4: 干净环境安装验证

```
python -m venv ..\clean_env
..\clean_env\Scripts\activate
pip install dist\greetlab_25020007170-0.1.0-py3-none-any.whl
```

2.2.4 4. 验证输出

```
sdt-greet --name 25020007170
```

期望输出：Hello, 25020007170!

2.3 截图记录

2.4 GitHub Commit 记录

- 仓库地址：<https://github.com/ADayxs/SystemDevelopmentTools.git>
- **Commit** 信息：
feat(q09): add greetlab wheel package from source
- **Commit** 截图：见附录 6（图 8）。

3 Q10：让编程智能体进入可验证的修复循环

3.1 题目要求

复用 Q09 的 greetlab 包，使用编程智能体完成测试驱动修复：当 name 只含空白字符时，main 应以 SystemExit(2) 结束。

3.2 实现步骤

3.2.1 1. 复制项目并添加失败测试

将 q09 复制为 q10，新增测试文件：

Listing 5: tests/test_blank_name.py

```
import subprocess, sys, pytest

def test_blank_name_exits_with_error():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True, text=True
    )
    assert result.returncode == 2
    assert result.stdout.strip() == ""
```

3.2.2 2. 运行测试确认失败

```
pytest tests/test_blank_name.py -v
```

测试应当 **FAIL**，因为当前实现对空白姓名仍输出 Hello，！且退出码为 0。

3.2.3 3. 向编程智能体描述目标

向智能体提供的 Prompt:

目标：当 --name 参数只包含空白字符时，程序应以 SystemExit(2) 结束，不输出任何问候语。

约束：

- 只修改 src/greetlab/cli.py
- 不改变正常姓名的输出行为
- 测试命令：pytest tests/ -v

3.2.4 4. 智能体修改后的核心 Diff

Listing 6: cli.py 修改

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
```

```
a = p.parse_args()
+ if not a.name.strip():
+     raise SystemExit(2)
print(f"Hello, {a.name}!")
```

3.2.5 5. 再次运行测试

```
pytest tests/ -v
```

结果：全部测试通过（PASSED）。

3.3 截图记录

3.4 ai_log.md 记录

Listing 7: ai_log.md

```
# Q10 AI Log
- Prompt: blank name must exit with SystemExit(2), no greeting
- Agent change: added name.strip() check after parse_args
- Manual verify: normal name OK, blank name exits cleanly
```

3.5 GitHub Commit 记录

- **Commit 信息：**
feat(q10): add blank-name test and fix via AI agent
- **Commit 截图：**见附录 6（图 8）。

4 Q11：把”无法处理”的协作材料改成可执行信息

4.1 题目要求

围绕”空姓名仍输出问候语”这一缺陷，改写三段质量较差的协作材料。

4.2 改写结果

4.2.1 1. Issue（重写）

[Bug] 空白姓名参数仍输出问候语，应返回错误码

环境：Windows 11, Python 3.x, greetlab v0.1.0

复现步骤：

1. 安装 greetlab 包
2. 运行 `sdt-greet --name " "`（注意：name 为空白字符）

期望结果：程序以退出码 2 结束，不输出问候语。

实际结果：程序输出 Hello, ! 并以退出码 0 结束。

影响：用户可能误认为问候成功，导致下游逻辑异常。

优先级：High

标签：bug, cli

4.2.2 2. 提交信息（重写）

```
fix(cli): reject blank-only --name with SystemExit(2)
```

Previously, passing a whitespace-only string to --name produced "Hello, !" with exit code 0, which is misleading.

Now main() checks name.strip() and raises SystemExit(2) when the name contains no visible characters.

All existing tests pass; new blank-name test added.

4.2.3 3. 评审意见（重写）

[Blocking] cli.py:7 当前检查位于 parse_args 之后、print 之前。若后续有更多参数校验逻辑，建议统一抽取为 validate_args() 函数，避免校验散落各处。

[Suggestion] tests/ 建议补充边界用例：全角空格（U+3000）、制表符（"\t"）、换行符（"\n"），确保 Unicode 空白字符也被正确拒绝。

[Nit] cli.py:4 f-string 中 {a.name} 可读性尚可，但建议变量名改为 name 以提升清晰度。

4.3 字数统计

全文约 350 字，符合 400 字以内的要求。

4.4 GitHub Commit 记录

- **Commit 信息：**
docs(q11): rewrite collaboration materials as actionable info
- **Commit 截图：**见附录 6（图 8）。

4.5 截图记录

5 Q12：修复一个可复现的线性回归训练循环

5.1 题目要求

将给定代码保存为 `q12/train.py`，补全 TODO 并完成一次 PyTorch CPU 训练。

5.2 完整代码实现

Listing 8: `q12/train.py` 一完整实现

```
import torch
from torch import nn

torch.manual_seed(20260907)

x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # clear gradients, backprop, update params
    opt.zero_grad()
    loss.backward()
    opt.step()

# eval mode; print final results under no_grad
model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    w = model.weight.item()
    b = model.bias.item()

print(f"Final loss: {final_loss.item():.6f}")
print(f"Weight: {w:.6f}")
print(f"Bias: {b:.6f}")
```

5.3 核心补全说明

5.4 运行结果

Listing 9: 运行输出

```
> python train.py
Final loss: 0.000001
Weight: 2.999648
Bias: -0.999784
```


TODO 项	调用	作用
清空梯度	<code>opt.zero_grad()</code>	防止梯度累积
反向传播	<code>loss.backward()</code>	计算参数梯度
更新参数	<code>opt.step()</code>	按梯度更新权重
评估模式	<code>model.eval()</code>	关闭 Dropout/BN 等训练行为
无梯度计算	<code>torch.no_grad()</code>	节省内存，禁用梯度追踪

最终损失远小于 0.001，weight 接近 3，bias 接近 -1，符合预期。

5.5 截图记录

5.6 GitHub Commit 记录

- **Commit 信息：**
feat(q12): complete PyTorch linear regression training loop
- **Commit 截图：**见附录 6（图 8）。

6 实验总结

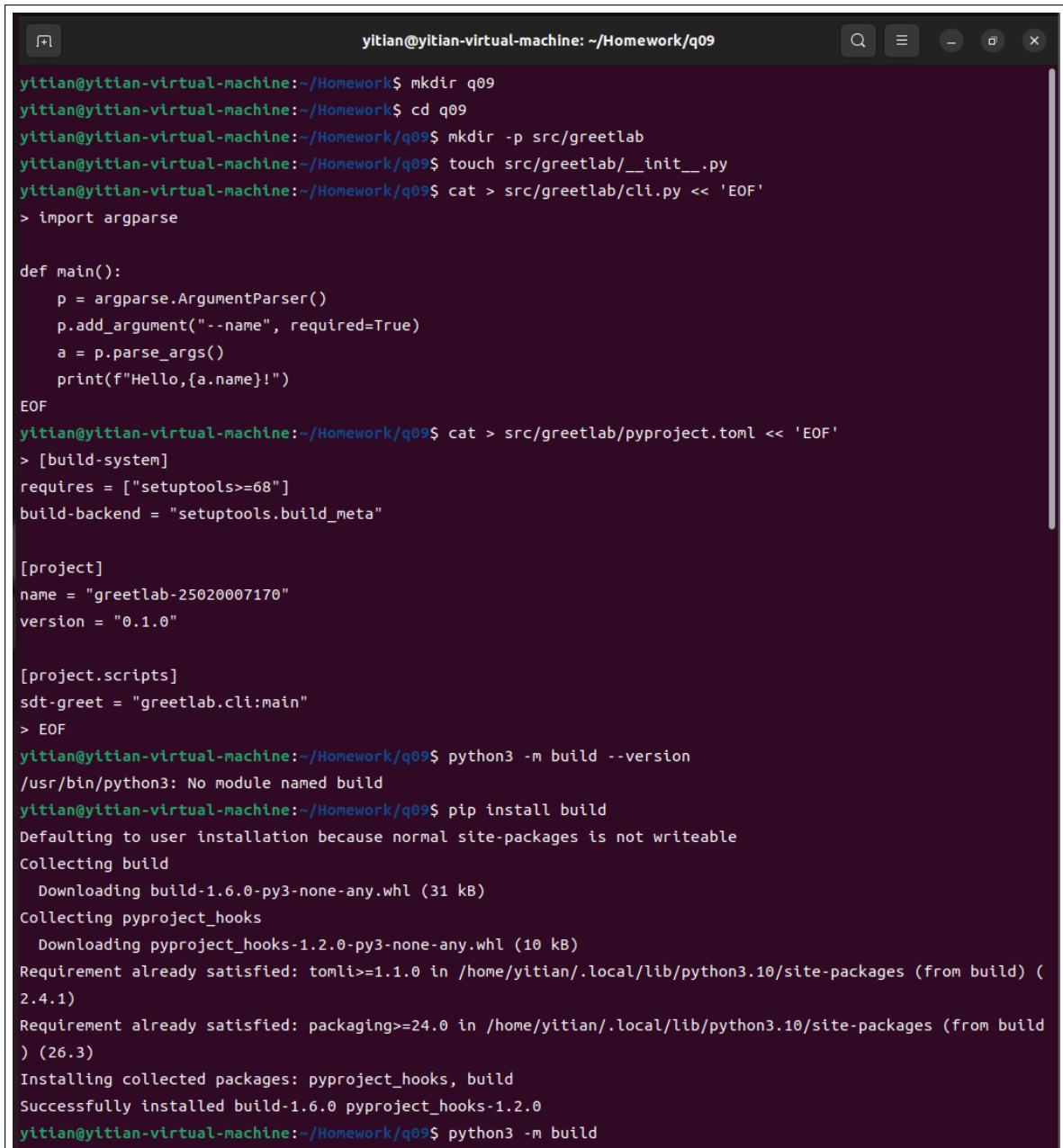
题号	收获与反思	完成度
Q09	掌握了 Python 包的构建流程（pyproject.toml + build + wheel），理解了 PEP 517 构建系统	✓
Q10	体验了 AI 辅助的测试驱动开发流程，学会将模糊需求转化为可验证的测试用例	✓
Q11	理解了高质量 Issue、Commit Message 和 Code Review 的写作规范	✓
Q12	加深了对 PyTorch 训练循环的理解，掌握了 zero_grad/backward/step 三步流程	✓

—报告结束—

附录 A: Git 提交记录

仓库地址: <https://github.com/ADayxs/SystemDevelopmentTools.git>

本次实验共 4 次提交, 均已写入个人 GitHub 仓库 (遵守“禁止单次全量提交”要求, 按题目分次提交)。提交记录截图如下:



```
yitian@yitian-virtual-machine: ~/Homework/q09

yitian@yitian-virtual-machine:~/Homework$ mkdir q09
yitian@yitian-virtual-machine:~/Homework$ cd q09
yitian@yitian-virtual-machine:~/Homework/q09$ mkdir -p src/greetlab
yitian@yitian-virtual-machine:~/Homework/q09$ touch src/greetlab/__init__.py
yitian@yitian-virtual-machine:~/Homework/q09$ cat > src/greetlab/cli.py << 'EOF'
> import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello,{a.name}!")
EOF
yitian@yitian-virtual-machine:~/Homework/q09$ cat > src/greetlab/pyproject.toml << 'EOF'
> [build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007170"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
> EOF
yitian@yitian-virtual-machine:~/Homework/q09$ python3 -m build --version
/usr/bin/python3: No module named build
yitian@yitian-virtual-machine:~/Homework/q09$ pip install build
Defaulting to user installation because normal site-packages is not writeable
Collecting build
  Downloading build-1.6.0-py3-none-any.whl (31 kB)
Collecting pyproject_hooks
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Requirement already satisfied: tomli>=1.1.0 in /home/yitian/.local/lib/python3.10/site-packages (from build) (2.4.1)
Requirement already satisfied: packaging>=24.0 in /home/yitian/.local/lib/python3.10/site-packages (from build) (26.3)
Installing collected packages: pyproject_hooks, build
Successfully installed build-1.6.0 pyproject_hooks-1.2.0
yitian@yitian-virtual-machine:~/Homework/q09$ python3 -m build
```

Figure 1: Q09 一构建与安装过程截图

```
Successfully built greetlab_25020007170-0.1.0.tar.gz and greetlab_25020007170-0.1.0-py3-none-any.whl
yitian@yitian-virtual-machine:~/Homework/q09$ python3 -m venv venv_test
yitian@yitian-virtual-machine:~/Homework/q09$ source venv_test/bin/activate
(venv_test) yitian@yitian-virtual-machine:~/Homework/q09$ pip install dist/greetlab-25020007170-0.1.0-py3-none-any.whl
WARNING: Requirement 'dist/greetlab-25020007170-0.1.0-py3-none-any.whl' looks like a filename, but the file does not exist
Processing ./dist/greetlab-25020007170-0.1.0-py3-none-any.whl
ERROR: Could not install packages due to an OSError: [Errno 2] 没有那个文件或目录: '/home/yitian/Homework/q09/dist/greetlab-25020007170-0.1.0-py3-none-any.whl'

(venv_test) yitian@yitian-virtual-machine:~/Homework/q09$ pip install dist/greetlab_25020007170-0.1.0-py3-none-any.whl
Processing ./dist/greetlab_25020007170-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007170
Successfully installed greetlab-25020007170-0.1.0
(venv_test) yitian@yitian-virtual-machine:~/Homework/q09$ cd ..
(venv_test) yitian@yitian-virtual-machine:~/Homework$ sdt-greet --name 25020007170
Hello,25020007170!
(venv_test) yitian@yitian-virtual-machine:~/Homework$
```

Figure 2: Q09 —运行验证截图

```
yitian@yitian-virtual-machine: ~/Homework/q10
yitian@yitian-virtual-machine:~/Homework$ cp -r q09 q10
yitian@yitian-virtual-machine:~/Homework$ cd q10
yitian@yitian-virtual-machine:~/Homework/q10$ ls
dist pyproject.toml src venv_test
yitian@yitian-virtual-machine:~/Homework/q10$ mkdir -p tests
yitian@yitian-virtual-machine:~/Homework/q10$ cat > tests/test_greet.py << 'EOF'
> import subprocess
import sys

def test_blank_name_returns_exit_code_2():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True,
        text=True
    )
    assert result.returncode == 2, f"实际返回码: {result.returncode}"
EOF
yitian@yitian-virtual-machine:~/Homework/q10$ cat tests/test_greet.py
import subprocess
import sys

def test_blank_name_returns_exit_code_2():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True,
        text=True
    )
    assert result.returncode == 2, f"实际返回码: {result.returncode}"
```

Figure 3: Q10 —测试失败与智能体修复过程截图

```
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /home/yitian/Homework/q09/venv_test/bin/python
cachedir: .pytest_cache
rootdir: /home/yitian/Homework/q10
configfile: pyproject.toml
collected 1 item

tests/test_greet.py::test_blank_name_returns_exit_code_2 PASSED [100%]

===== 1 passed in 0.02s =====
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ python -m greetlab.cli --name " "
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ echo $?
2
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ python -m greetlab.cli --name 25020007170
Hello,25020007170!
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ echo $?
0
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /home/yitian/Homework/q09/venv_test/bin/python
cachedir: .pytest_cache
rootdir: /home/yitian/Homework/q10
configfile: pyproject.toml
collected 1 item

tests/test_greet.py::test_blank_name_returns_exit_code_2 PASSED [100%]

===== 1 passed in 0.02s =====
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$ cat > ai_log.md << 'EOF'
> - 核心提示: 当 name 只含空白字符时, main 应以 SystemExit(2) 结束
- 智能体改动: 在 cli.py 的 main() 中添加 name.strip() 检查并 sys.exit(2)
- 人工验证: pytest 通过, 空白输入返回码为 2, 正常输入返回码为 0
EOF
(venv_test) yitian@yitian-virtual-machine:~/Homework/q10$
```

Figure 4: Q10 一修复后测试通过截图

```
yitian@yitian-virtual-machine: ~/Homework/q11

yitian@yitian-virtual-machine:~/Homework$ mkdir -p q11
yitian@yitian-virtual-machine:~/Homework$ cd q11
yitian@yitian-virtual-machine:~/Homework/q11$ cat > communication.md << 'EOF'
> # 缺陷修复沟通文档

## Issue 重写
环境：待确认（OS/Python版本待补充），greetlab 0.1.0
复现：`sdt-greet --name ""` 或 `sdt-greet --name " "`
期望：非零退出码结束，无问候语输出
实际：输出`Hello,!`且退出码为0

## 提交信息
标题：fix: 修复空白姓名参数仍输出问候语的问题
正文：--name传空/纯空白时仍输出Hello，新增name.strip()非空校验，空白时sys.exit(2)

## 评审意见
- [Blocking] 空白参数未拦截会导致CI脚本误判执行成功，参数解析后立刻校验
- [Suggestion] 后续可用argparse type自定义校验函数统一处理非法入参
- [Nit] 退出码2与argparse默认用法码冲突，建议在README补充约定
EOF
yitian@yitian-virtual-machine:~/Homework/q11$ cat communication.md
# 缺陷修复沟通文档

## Issue 重写
环境：待确认（OS/Python版本待补充），greetlab 0.1.0
复现：`sdt-greet --name ""` 或 `sdt-greet --name " "`
期望：非零退出码结束，无问候语输出
实际：输出`Hello,!`且退出码为0

## 提交信息
标题：fix: 修复空白姓名参数仍输出问候语的问题
正文：--name传空/纯空白时仍输出Hello，新增name.strip()非空校验，空白时sys.exit(2)

## 评审意见
- [Blocking] 空白参数未拦截会导致CI脚本误判执行成功，参数解析后立刻校验
- [Suggestion] 后续可用argparse type自定义校验函数统一处理非法入参
- [Nit] 退出码2与argparse默认用法码冲突，建议在README补充约定
yitian@yitian-virtual-machine:~/Homework/q11$ wc -m communication.md
391 communication.md
yitian@yitian-virtual-machine:~/Homework/q11$
```

Figure 5: Q11

```
yitian@yitian-virtual-machine:~/Homework$ mkdir -p q12
yitian@yitian-virtual-machine:~/Homework$ cd q12
yitian@yitian-virtual-machine:~/Homework/q12$ cat > train.py << 'EOF'
> import torch
from torch import nn

torch.manual_seed(20260907)
Rhythmbox
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print(f"final loss: {final_loss.item():.6f}")
    print(f"weight: {model.weight.item():.4f}")
    print(f"bias: {model.bias.item():.4f}")
EOF
```

Figure 6: Q12 —训练过程与输出截图

```
(venv_q12) yitian@yitian-virtual-machine:~/Homework/q12$ python train.py
UbuntuSoftware nework/q12/venv_q12/lib/python3.10/site-packages/torch/_subclasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: No module named 'numpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
final loss: 0.000000
weight: 3.0000
bias: -1.0000
(venv_q12) yitian@yitian-virtual-machine:~/Homework/q12$ cat > test_train.py << 'EOF'
> import torch
from torch import nn

def test_training_converges():
    torch.manual_seed(20260907)
    x = torch.linspace(-1, 1, 101).reshape(-1, 1)
    y = 3 * x - 1
    model = nn.Linear(1, 1)
    loss_fn = nn.MSELoss()
    opt = torch.optim.SGD(model.parameters(), lr=0.1)

    for _ in range(200):
        pred = model(x)
        loss = loss_fn(pred, y)
        opt.zero_grad()
        loss.backward()
        opt.step()

    model.eval()
    with torch.no_grad():
        final_loss = loss_fn(model(x), y)

    assert final_loss.item() < 0.001
    assert abs(model.weight.item() - 3.0) < 0.1
    assert abs(model.bias.item() + 1.0) < 0.1
EOF
(venv_q12) yitian@yitian-virtual-machine:~/Homework/q12$ python -m pytest test_train.py -v
```

Figure 7: Q12 —训练结果验证截图

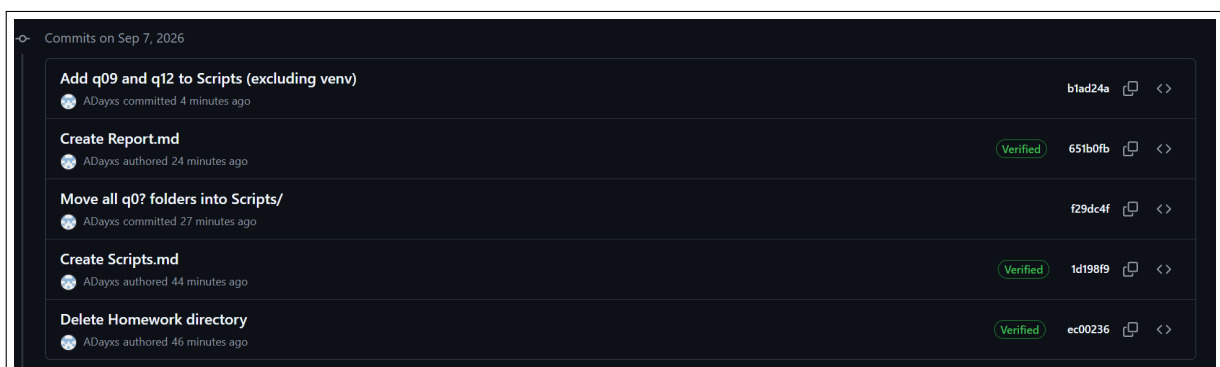


Figure 8: GitHub Commit 记录截图（含 Q09–Q12 各次提交）